

COMS3011A Project 3

Brendan Griffiths

1 Sport Analytics Tool

Large leagues keep databases to track how their competitors perform. Unsatisfied with what (insert favourite sport here) currently offers, you have decided to build one of your own for a professional league or tour. The platform needs to let approved users submit and manage the statistics of its competitors, whether those are individual athletes or teams, and to provide an API for other platforms to use. Every statistic that matters for the sport should be available, and preferably calculated from event data rather than manual entry. The platform should serve as a home for anyone wanting to do large-scale analysis of the sport, and should offer the APIs and datasets to support that work.

1.1 Features

1.1.1 Basic

The platform should be built on event data. Every statistic it publishes should be derived from a record of the individual events that occurred during a single fixture, whether that is a match, a race, or a heat, in the order they occurred, rather than stored as a total that somebody typed in. This derivation is the core of the platform, and the submissions, the API, and the datasets are all arranged around it. Which events are worth recording, and which statistics the league expects computed from them, depends on the sport the group builds for. Correcting an event should be enough to bring every statistic derived from that event back in line, without anything being re-entered by hand.

Event data reaches the platform by submission. Only approved submitters should be able to submit, and each should submit within a defined part of the competition, so it is always clear who supplied a given event. A submission, uploaded as a file or sent to the platform directly, should be checked against the platform's event schema before it is accepted, and a rejection should tell the submitter what was wrong with it rather than failing quietly. Every published statistic should be traceable back to the events and the submission behind it.

The API is the platform's product rather than an accessory to a web interface. Another platform should be able to read fixtures, events, and derived statistics through it, narrow a request to the slice it cares about, and page through results too large to return at once. Identifiers should be stable, so that a consumer holding a reference from last season can still resolve it. An analyst should also be able to export a filtered slice of the data as a file for use elsewhere.

1.1.2 Intermediate

Submitting one fixture at a time should become a pipeline. Whole seasons and back catalogues arrive at once, so a batch should be staged and validated before it lands, and should report what it did and did not

accept. Resubmitting a batch should not double-count anything, and a batch that fails part way through should resume rather than restart. Submissions should pass a review before publication, with validation rules that catch impossible or conflicting data, and corrections should leave a history behind them.

The derivation should reach beyond a single fixture, to season, career, and competition-wide aggregates. At this size, recomputing every statistic from scratch stops being viable, so a change to the event data should only cause the figures that depend on it to be recomputed. Figures should be checked against reference results already known to be right, rather than assumed correct because they were computed.

The platform should be exercised at the size it is meant to serve: hundreds of fixtures, each with hundreds of events. Queries should meet a stated response time under that load, achieved through the schema, indexing, and storage layout rather than by chance. The API should be versioned, should issue keys to its consumers, and should hold them to rate limits and quotas so that one consumer cannot exhaust the platform for the rest, with repeated reads served from cache where they can be.

The datasets should become releases rather than ad-hoc downloads: versioned snapshots published with their schema, a description of every field, and a checksum, so an analysis run against a release can be repeated later and produce the same numbers.

1.1.3 Advanced

The derivation layer should stop being fixed. An analyst should be able to define a new statistic over the event schema itself, and have the platform evaluate that definition across the entire history at the same scale as its built-in statistics. Definitions should be validated before they run, contained so a bad one cannot damage the platform, and versioned, so a figure published under one definition stays reproducible after it changes. This is the hardest part of the platform: a derived statistic is only worth anything if it is correct, and it has to stay correct while the events underneath it are still being corrected.

Where submission was previously a batch after the event, the platform should also accept a feed from a fixture in progress, and should cope with events that arrive late or out of order, so that a disordered feed still settles on the same statistics as the same events submitted in order. The pipeline should be replayable, and any figure the platform publishes should be traceable to the events and the definition version that produced it.

Because corrections keep arriving, the platform should be able to say what a statistic was as of a given date rather than only what it is now, and should let a consumer see what changed between two dataset releases. The API should serve aggregate questions and not only records, hand large requests off as jobs a consumer collects once they are ready, and offer a feed of changes so a consumer already holding a release can pull only the difference.

The API should also be able to change without breaking the platforms depending on it, retiring versions along a published deprecation path, testing its own contracts, and showing each consumer what it has used. Finally, the platform should defend its own numbers, flagging events that look wrong against the history, reconciling submitters that disagree, and carrying an accepted correction through to every aggregate and release downstream of it.

AI Declaration: The preceding document was generated with: Claude-Code[Claude Opus 4.8]